Project - Low Level Design
On
Project Title

IaC Provisioning for Real Estate System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Srno	StudentName	EnrolmentNumber
1	Cherry Mehta	EN22CS301292
2	Bhumika Choyal	EN22CS301276
3	Ayush Jain	EN22CS301247
4	Ayush Patidar	EN22CS301253
5	Diya Goyal	EN22CS301351

Group Name:03D3

Project Number:DO-27

Industry Mentor Name: Prof Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-26

1. Introduction

The Low Level Design (LLD) describes the internal working of the **IaC-based provisioning system** for a Real Estate application. The system automates infrastructure setup using DevOps tools such as:

- Terraform (Infrastructure provisioning)
- Docker (Containerization)
- Jenkins / GitHub Actions (CI/CD automation)
- Kubernetes (Container orchestration)

The system ensures:

- Automated infrastructure deployment
- Reduced manual configuration errors
- Scalable and repeatable environments
- Faster application deployment

The architecture follows a modular design for better maintainability and automation.

1.1 User Interface Module (Dashboard / API Layer)

Technology Used: CLI / Web Dashboard

Responsibilities:
- Accept deployment requests
- Trigger infrastructure provisioning
- Show deployment status
- Display logs and errors

Inputs:
- Deployment command / trigger
- Configuration files

Outputs:
- Deployment status (Success/Failed)
- Logs and error messages

1.2 Configuration Management Module

Purpose:
Manages all infrastructure and deployment configurations.

Responsibilities:

- Store Terraform scripts
- Manage Ansible playbooks (if used)
- Maintain environment variables

Validations Performed

- Syntax validation of scripts
- Configuration consistency check

Outcome

- Valid configuration files
- Ready-to-deploy infrastructure setup

1.3 Infrastructure Provisioning Module (Terraform)

Purpose:
Automates cloud/local infrastructure creation.

Responsibilities:

- Create EC2 / VM instances
- Setup networking (VPC, Subnets)
- Allocate storage

Core Methods:
terraform plan
terraform apply
terraform destroy

1.4 Containerization Module (Docker)

Purpose:
Packages application into containers.

Responsibilities:

- Create Docker images
- Manage Docker containers
- Ensure environment consistency

Core Methods:
Dockerfile
Docker Image
Docker Container

1.5 CI/CD Pipeline Module (Jenkins / GitHub Actions)

Purpose:
Automates build, test, and deployment pipeline.

Responsibilities
- Pull code from Git
- Build Docker images
- Run tests
- Deploy application

Pipeline Stages
- Source → Build → Test → Deploy

1.6 Orchestration Module (Kubernetes)

Purpose
Manages container deployment and scaling.

Responsibilities
- Deploy containers (Pods)
- Load balancing (Services)
- Auto-scaling

Core Components
- Pods
- Deployments
- Services

1.7 Workflow Orchestration Module

Purpose:
Tracks system performance and logs.

Responsibilities
- Monitor container health
- Track deployment logs
- Alert on failures

2. Class Design

Class: UserInterface

Methods:

- trigger_deployment()
- view_logs()
- show_status()

Class: ConfigManager
Methods:
- load_config()
- validate_config()

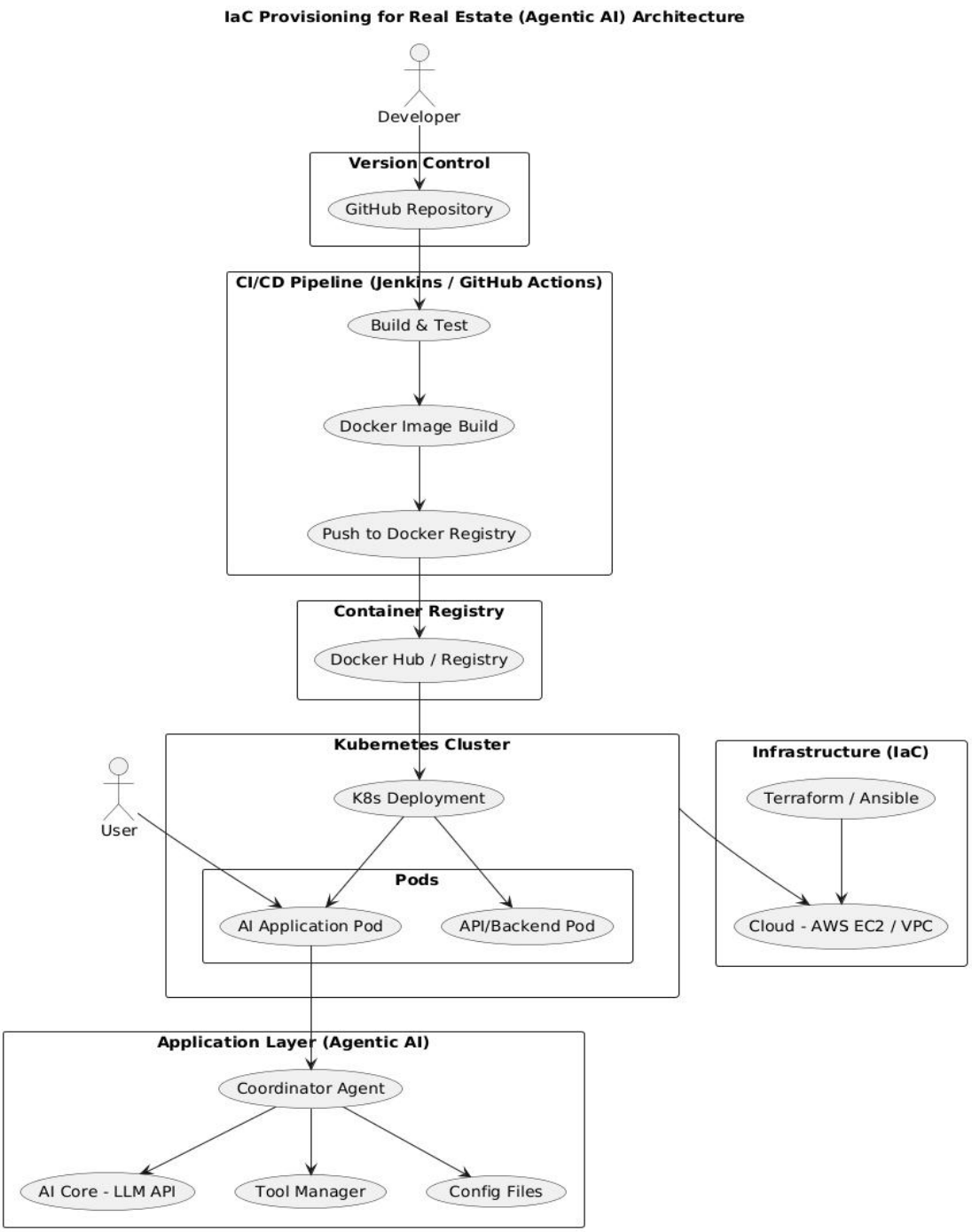
Class: TerraformManager
Attributes:
- config_path

Methods:
- init()
- plan()
- apply()
- destroy()

Class: DockerManager
Methods:
- build_image()
- run_container()
- push_image()

Class: CICDPipeline
Methods:
- trigger_pipeline()
- build()
- deploy()

Class: Orchestrator
Methods:
- start_pipeline()
- handle_errors()
- manage_workflow()



3. Detailed Process Flow

- User triggers deployment
- Configuration module validates scripts
- Terraform provisions infrastructure

- Docker builds application image
- CI/CD pipeline triggers automation
- Kubernetes deploys containers
- Monitoring module tracks performance